

THURSDAY, AUGUST 13TH, 2026

OPERATIONS RESEARCH

Linear Programming Duality and the Shadow Prices Hidden in Every Optimal Plan

How a Leningrad plywood trust, a wartime diet problem, and a Bell Labs patent fight built the quiet arithmetic behind every refinery schedule, airline roster, and ad auction running today.

2,743 words · 14-minute read

In the fall of 1947, nine clerks sat down at hand-cranked desk calculators in the U.S. National Bureau of Standards and started multiplying. Their task was to find the cheapest possible diet — the exact combination of 77 foods, from wheat flour to beef liver, that supplied every nutrient a body needs at the lowest annual cost. The economist George Stigler had already guessed at an answer in 1945 using intuition and some clever approximation: \$39.93 a year, in 1939 prices [1]. It took the clerks roughly 120 man-days of pencil-and-crank human labor to prove that the true minimum was \$39.69. Stigler's guess had missed the mathematically perfect diet by twenty-four cents [1].

That absurd-sounding exercise is a founding legend of a field that now quietly runs an enormous fraction of the physical economy. The clerks were the first humans to execute by hand an algorithm called the simplex method — invented a few months earlier by the mathematician George Dantzig — on a real, messy, economically meaningful problem. What they proved is that somewhere among the billions of admissible diets there is one single best one, and that a systematic procedure, not guesswork, finds it exactly. That procedure, and the duality theory built on top of it, now schedules oil refineries, air crews, power grids and shipping fleets, and is being rebuilt to run on GPUs at a scale of billions of variables.

Two Men, Two Sides of an Iron Curtain

The strange thing about linear programming's origin story is that it happened twice, independently, by people who could not talk to each other.

In 1939 a young Soviet mathematician named Leonid Kantorovich was asked by a Leningrad plywood trust how to allocate production across eight machines to maximize output, given that each machine had a different efficiency for each of the trust's product lines. He recognized the structure of dozens of other problems he had been handed — allocating raw materials, transportation and labor across competing uses — and developed a general method he called the method of resolving multipliers [2]. It is, in modern language, linear programming. The war forced him out of the siege of Leningrad, and he continued the work attached to the Soviet Navy's engineering school. When he argued that his pricing-

like multipliers could rationalize Soviet resource allocation, ideological critics condemned the idea as anti-Marxist — prices, in a planned economy, were supposed to be politically determined, not computed [2]. His work sat largely unknown in the West until 1956, when the American economist Tjalling Koopmans arranged for it to be translated and published in *Management Science* [2].

Meanwhile, in 1947, Dantzig was working for the U.S. Air Force on Project SCOOP, an effort to mechanize the planning of wartime logistics: which units get which supplies, routed by which means, on what schedule. He took Wassily Leontief's input-output model of the economy and generalized it — instead of one fixed recipe per good he allowed alternative activities, and instead of merely describing the economy he added an objective, minimize cost or maximize output. He then needed an algorithm to solve the resulting system, and when the literature turned up nothing, he built one: the simplex method [1].

Dantzig and Kantorovich had, under wildly different political systems, invented the same machinery for the same reason: both served governments at war, trying to squeeze the most output out of scarce, unevenly capable resources. In 1975 the Royal Swedish Academy of Sciences awarded the Nobel Memorial Prize in Economic Sciences jointly to Kantorovich and Koopmans “for their contributions to the theory of optimum allocation of resources” [3] — a quiet bridge across the Iron Curtain, built out of an algorithm neither side's ideology was fully comfortable with.

What a Linear Program Actually Is

Strip away the history and a linear program is a simple object. You have a vector of decision variables $\mathbf{x} = (x_1, \dots, x_n)$ you are allowed to choose, subject to linear inequality constraints, and you want to optimize a linear objective:

$$\text{maximize } z = \mathbf{c}^T \mathbf{x} \quad \text{subject to } \mathbf{A}\mathbf{x} \leq \mathbf{b}, \mathbf{x} \geq \mathbf{0}$$

Every constraint $\mathbf{A}_i \mathbf{x} \leq b_i$ is a half-space: everything on one side of a flat boundary. Intersect enough half-spaces and you get a convex polytope, a many-cornered solid — in two dimensions, a polygon — containing every combination of decisions that breaks no rule. That is the feasible region. Because the objective is linear, with no curvature to trap you in a local dip, a basic fact of geometry guarantees that if an optimum exists, at least one is achieved at a vertex, a literal corner of that polytope. You never need to search the vast continuous interior; the best plan always lives on the boundary, at a point where several constraints meet at once.

This is the intuition behind the simplex method: start at any vertex, then repeatedly ask whether some adjacent vertex, one edge away, is better. If yes, walk to it. If no neighbor improves the objective, you are done — because the region is convex, a vertex with no better neighbor is the global optimum, not merely a local one.

What makes this extraordinary is scale. A refinery LP might have tens of thousands of variables; an airline's crew-scheduling LP can have millions. The polytope in such problems has more vertices than a grain of sand has atoms, yet simplex routinely finds the optimal one in seconds, walking a short improving path across the surface and never visiting more than a tiny fraction of them. Why that works so well is a scientific mystery in its own right.

The Twin Problem Hiding Inside Every LP

Every linear program has a shadow twin. Take the maximization problem above, associate one non-negative variable y_i with each constraint, and build the *dual* program:

$$\text{minimize } w = b^T y \quad \text{subject to } A^T y \geq c, \quad y \geq 0$$

The weak duality theorem says that for any feasible x in the primal and any feasible y in the dual, $c^T x \leq b^T y$ — every dual solution is an upper bound on every primal one, and vice versa [7]. The strong duality theorem, considerably deeper and the real engine of the theory, says that if the primal has a finite optimum then so does the dual, and the two optimal values are exactly equal: $c^T x^* = b^T y^*$ [7]. Solve one problem and the other comes free.

CHECK THIS AGAINST YOUR INTUITION

The dual has one variable per primal constraint, and its objective is built from the right-hand sides b rather than the costs c . Before reading on: what quantity, in units, would a variable attached to a capacity constraint have to be measuring?

Units answer the question. If constraint i is measured in barrels per day and the objective in dollars per day, then y_i must carry dollars per barrel — it is a price. That is what the dual variables are: y_i^* is the *shadow price* of constraint i , the marginal value of one more unit of whatever resource that constraint limits [7]. If a constraint caps a distillation unit's throughput at 100,000 barrels a day, its shadow price says how many additional dollars of margin one more barrel of capacity would earn — the quantity that decides whether a debottlenecking project pays for itself. If a constraint has slack, its shadow price is zero, by a companion result called complementary slackness: an unused resource cannot be limiting profit, so relaxing it further is worth nothing. Economists will recognize these variables as Lagrange multipliers wearing a different hat, and as a discrete cousin of the market-clearing prices of Walrasian general equilibrium. That is precisely why Kantorovich's resolving multipliers unsettled Soviet orthodoxy: they were, mathematically, prices — emerging from an optimization problem rather than from a market.

A Refinery in Miniature

Consider a simplified refinery deciding how many thousand barrels a day of two crude types to process. Crude A is light and easy to work with but supply-limited; Crude B is heavier and more available, but contributes more sulfur to the blend. Let x_1, x_2 be thousands of barrels per day of each crude, with margin in thousands of dollars per day:

$$\begin{aligned} \text{maximize } z &= 8x_1 + 5x_2 \\ x_1 + x_2 &\leq 100 && \text{(distillation capacity)} \\ x_1 &\leq 60 && \text{(Crude A allocation)} \\ 0.3x_1 + 0.8x_2 &\leq 60 && \text{(sulfur budget)} \\ x_1, x_2 &\geq 0 \end{aligned}$$

The feasible region is a pentagon with corners at the origin, $(60, 0)$, $(60, 40)$, $(40, 60)$ and $(0, 75)$, and the optimum — as the geometry promises — sits at one of them.

Vertex (x_1, x_2)	Binding constraints	$z = 8x_1 + 5x_2$
$(0, 0)$	both non-negativity bounds	0
$(60, 0)$	Crude A cap, $x_2 \geq 0$	480
$(60, 40)$	Crude A cap, capacity	680
$(40, 60)$	capacity, sulfur budget	620
$(0, 75)$	sulfur budget, $x_1 \geq 0$	375

Table 1 — Objective value at each corner of the feasible polygon.

The winner runs Crude A at its full allocation and fills the rest of the distillation column with Crude B, for a margin of \$680,000 a day. It finishes ten sulfur-budget units under the limit: $0.3(60) + 0.8(40) = 50$. The sulfur specification, the constraint an engineer might have assumed was binding, never binds at all.

Solving the dual confirms the shadow-price story. Let y_1, y_2, y_3 price capacity, Crude A supply and the sulfur budget. Complementary slackness forces $y_3 = 0$, since the sulfur constraint has slack. Both crudes are produced in positive amounts, so both dual constraints hold with equality, leaving $y_1 + y_2 = 8$ and $y_1 = 5$. The dual objective is then $100(5) + 60(3) + 60(0) = 680$, landing exactly on the primal optimum: strong duality, verified by hand.

Quantity	Value	Interpretation
x_1^* (Crude A)	60 kbbl/d	run at full allocation
x_2^* (Crude B)	40 kbbl/d	fills remaining capacity
y_1^* (capacity)	\$5,000/kbbl	value of one more kbbl/d of throughput
y_2^* (Crude A)	\$3,000/kbbl	value of one more kbbl/d of Crude A
y_3^* (sulfur)	0	slack constraint — worthless to relax

Table 2 — Primal solution and its dual shadow prices.

The prices are checkable by brute force. Raise capacity to 101 and the optimum becomes **(60, 41)**, worth 685 — five more, exactly y_1 . Raise the Crude A allocation to 61 and the optimum becomes **(61, 39)**, worth 683 — three more, exactly y_2 . Loosen the sulfur budget and nothing happens at all. Two of the three resources have a defensible internal price; the third, despite dominating the engineering conversation, is worth nothing at the margin — and none of it came from a separate study.

From Octane Giveaways to a Flagship Industry

This toy is a miniature of the problem that made linear programming an industrial discipline. In 1949, over lunch, the operations researchers W. W. Cooper and Abraham Charnes discussed with Gulf Oil the “octane giveaway” problem: refineries were routinely blending in more expensive, higher-octane components than customers paid for, because nobody had a systematic way to hit a specification exactly rather than generously. Cooper and Charnes formulated it as a linear program [8]. Esso Standard Oil followed in 1953 with a full gasoline-blending LP published by G. H. Symonds, and Magnolia Petroleum built an LP in 1957 for scheduling which crude sources to run to maximize profit across a whole system [8]. By 1955 a symposium of the Operations Research Society of America was showcasing LP applied to reservoir scheduling, product distribution and delivery routing. Petroleum had become the flagship industry proving that this new mathematics was not merely a wartime curiosity [8].

The Algorithm That Should Not Work So Well

Here the story takes a strange theoretical turn. In 1972 Victor Klee and George Minty published a nasty little counterexample: a perturbed hypercube, now called the Klee–Minty cube, with 2^n corners in n dimensions. On it, Dantzig’s standard pivot rule visits every one of those exponentially many vertices before finding the optimum [4]. The algorithm industry had been relying on for twenty-five years thus had no proof of ever being *fast*; in the worst case it was exponential, no better than brute-force search.

That mattered enormously in theoretical computer science, where polynomial time is the bar for calling an algorithm efficient. In 1979 the Soviet mathematician Leonid Khachiyan showed that a different technique — the ellipsoid method, which shrinks a sequence of ellipsoids around the feasible region

rather than walking its edges — solves any LP in provably polynomial time [9]. It made front-page news as a Cold War mathematical event. It then turned out to be nearly useless in practice: reliably, painfully slower than simplex on the problems industry actually cared about. Provably fast in theory and useless in practice; unproven in theory and reliably fast in practice. For over a decade, linear programming had two algorithms and neither satisfied both halves of that sentence.

WORK OUT WHICH SIDE OF THE TRADE YOU WOULD TAKE

Suppose a plant must be scheduled every morning, and exactly one of those two algorithms may be used for the next decade. Which guarantee is worth more: a proof of a worst-case bound, or twenty-five years of the thing never once having been slow?

Industry answered by keeping simplex, and industry was right — but for reasons nobody could articulate until much later. The intermediate resolution came in 1984, when Narendra Karmarkar, then at AT&T Bell Laboratories, published an interior-point method: a way of cutting straight through the middle of the feasible polytope along a curved path toward the optimum, rather than crawling along its boundary edge by edge [5]. It was provably polynomial — roughly $O(n^{3.5}L)$ arithmetic operations for n variables and L -digit data — *and*, unlike the ellipsoid method, fast enough in practice to matter commercially [5]. AT&T patented it and built dedicated optimization hardware called KORBX, priced at \$8.9 million, with the Pentagon as its first customer [5]: a striking measure of what a few percentage points of allocative efficiency was worth to a government at the height of the Cold War. The patent expired in 2006, and interior-point methods are now standard in every serious solver.

The deepest resolution came later still. In 2004 Daniel Spielman and Shang-Hua Teng introduced smoothed analysis. Instead of asking about the very worst possible input, which Klee–Minty shows is catastrophic, or the average random input, which is too optimistic to mean much, they asked what happens to *any* input after a small random perturbation — modeling the fact that real data always carries a little noise. Under such perturbations, they proved, simplex’s expected running time is polynomial [6]. The Klee–Minty cube is exponential precisely because it is adversarially exact; real capacities and real crude assays are never that perfectly tuned, and that imperfection is what saves the algorithm. It is a rare case of pure theory explaining, thirty years late, why an engineering success had always felt slightly miraculous.

From Nine Clerks to Twelve Billion Nonzeros

The arc from Stigler’s dining table to the present is worth holding in view all at once. Nine humans with desk calculators needed 120 man-days to optimize 77 variables. Today Google’s PDLP solver — a first-order method that, unlike simplex or interior-point methods, avoids matrix factorization entirely in favor of simple matrix-vector multiplication — has solved lower-bound instances of the traveling salesman problem with constraint matrices containing up to twelve billion nonzero entries [10]. Because it needs little memory and parallelizes well, it runs on GPUs and distributed clusters, and Google has deployed it

since 2023 to route traffic across entire data-center networks in production [10]. The same mathematical object Kantorovich sketched for a plywood trust under wartime siege now allocates bandwidth for some of the largest computing infrastructure ever built.

What remains open is whether the theory-practice gap that took thirty years to explain for simplex is quietly recurring for these first-order methods. They are fast and they scale, but the convergence guarantees interior-point methods enjoy do not transfer cleanly, and PDLP's speed on real, structured problems runs ahead of any complete account of why it is so reliably that fast. Each generation seems to buy its scale by taking on a new debt to theory, repaid a decade or two later by someone proving what practitioners already knew. Kantorovich might have found that a satisfying kind of continuity: the plywood trust's problem never went away, it just kept getting bigger.

REFERENCES

- [1] G. B. Dantzig, "The Diet Problem," *Interfaces*, vol. 20, no. 4, pp. 43–47, 1990. [Online]. Available: <https://gwern.net/doc/statistics/decision/stigler-diet/1990-dantzig.pdf>
- [2] INFORMS, "Kantorovich, Leonid V.," *History of O.R. Excellence — Biographical Profiles*. [Online]. Available: <https://www.informs.org/Explore/History-of-O.R.-Excellence/Biographical-Profiles/Kantorovich-Leonid-V>
- [3] The Nobel Prize, "The Sveriges Riksbank Prize in Economic Sciences in Memory of Alfred Nobel 1975," NobelPrize.org. [Online]. Available: <https://www.nobelprize.org/prizes/economic-sciences/1975/summary/>
- [4] V. Klee and G. J. Minty, "How Good is the Simplex Algorithm?," in *Inequalities III*, O. Shisha, Ed. New York: Academic Press, 1972, pp. 159–175.
- [5] N. Karmarkar, "A New Polynomial-Time Algorithm for Linear Programming," in *Proc. 16th Annual ACM Symposium on Theory of Computing*, 1984, pp. 302–311.
- [6] D. A. Spielman and S.-H. Teng, "Smoothed Analysis of Algorithms: Why the Simplex Algorithm Usually Takes Polynomial Time," *Journal of the ACM*, vol. 51, no. 3, pp. 385–463, 2004.
- [7] MIT 15.053, "Duality in Linear Programming," Course Notes, Chapter 4. [Online]. Available: <https://web.mit.edu/15.053/www/AMP-Chapter-04.pdf>
- [8] INFORMS, "Chemical and Petroleum," *History of O.R. Excellence — O.R. Application Areas*. [Online]. Available: <https://www.informs.org/Explore/History-of-O.R.-Excellence/O.R.-Application-Areas/Chemical-and-Petroleum>
- [9] "Leonid Khachiyan," Wikipedia. [Online]. Available: https://en.wikipedia.org/wiki/Leonid_Khachiyan
- [10] Google Research, "Scaling up linear programming with PDLP," research.google blog. [Online]. Available: <https://research.google/blog/scaling-up-linear-programming-with-pdlp/>

Daily Learning No. 001 · Operations Research · Thursday, August 13th, 2026 · Sources are listed above; every load-bearing figure traces to a numbered reference.